

A Linear Space Algorithm for the LCS Problem

S. Kiran Kumar^{*} and C. Pandu Rangan

Department of Computer Science and Engineering, Indian Institute of Technology,
Madras 600036, India

Summary. The LCS problem is to determine the longest common subsequence (LCS) of two strings. A new linear-space algorithm to solve the LCS problem is presented. The only other algorithm with linear-space complexity is by Hirschberg and has runtime complexity $O(mn)$. Our algorithm, based on the divide and conquer technique, has runtime complexity $O(n(m-p))$, where p is the length of the LCS.

I. Introduction

A subsequence of a string of symbols is obtained by deleting zero or more symbols from the string. Let A and B be two strings and their lengths be m and n respectively. A string C is a common subsequence of A and B if C is a subsequence of both A and B . C is a longest common subsequence (LCS) of A and B , denoted by $C = \text{LCS}(A, B)$, if

- (i) C is a common subsequence of A and B .
- (ii) No string longer than C is a common subsequence of A and B .

For example, LCS of the strings “printed” and “written” is “rite”. In general, two strings can have more than one LCS. As an example, both “ac” and “bc” are LCS’s of “abc” and “bac”.

Various solutions have been proposed for the LCS problem [3–6, 9–12]. Hirschberg presented a linear-space algorithm with time complexity $O(mn)$ in [3]. In [4], he proposed two quadratic-space algorithms whose time complexities are $O(np)$ and $O(p(m-p)\log n)$, where p is the length of the LCS. Hunt et al. [6] presented an algorithm whose time and space complexities are $O((r+n)\log n)$ and $O(r+n)$ respectively, where r is the number of matched pairs in both strings. Hsu [5] improved the runtime complexities of the $O(np)$ algorithm presented

^{*} Present address: Department of Computer Sciences, University of Texas at Austin, Austin, Texas 78712, USA

earlier by Hirschberg [4] and the $O(r+n)$ algorithm presented by Hunt [6]. Nakatsu [11] presented an $O(n(m-p))$ algorithm, but it requires quadratic space. We present an algorithm, based on the divide and conquer technique, with $O(n(m-p))$ time complexity and linear space complexity. Lower bounds for the LCS problem were studied by Aho [1]. They showed that any algorithm using “equal-unequal” comparisons must, in the worst case, take quadratic time or assume a fixed alphabet size.

The LCS problem has numerous applications. It has been used by molecular biologists to study amino acid sequences of similar proteins in different organisms and to trace the history of living beings [2]. Other applications are in data compression, where instead of storing two similar files, the LCS of the files along with the modifications for the individual files are stored [8]. (We can consider a file to be a string in which each line is treated as a symbol.) The LCS problem has applications in pattern recognition where the LCS of two strings is used as a certain similarity measure of the objects represented by the strings [7].

2. Basic Definitions and Theorems

Notation. We denote string A by $A(1)A(2)\dots A(m)$ and string B by $B(1)B(2)\dots B(n)$. A segment of A from the i^{th} symbol to the j^{th} symbol ($i \leq j$) is denoted by $A(i:j)$. That is, $A(i:j) = A(i)A(i+1)\dots A(j)$. $\hat{A}$ is used to denote the reverse of the string A or $A(m)A(m-1)\dots A(1)$. PQ denotes the catenation of the two strings P and Q . $|A|$ denotes the length of A .

Definition 1 [11]. $L_i(k)$ denotes the largest h such that $A(i:m)$ and $B(h:n)$ have a LCS of length k .

Definition 2. $L_i^*(k)$ denotes the smallest h such that $A(1:i)$ and $B(1:h)$ have a LCS of length k .

Obviously, $L_i(0) = n+1$ ($1 \leq i \leq m+1$), and $L_i^*(0) = 0$ ($0 \leq i \leq m$).

Each $L_i(k)$ can be considered as pointer that tells us how much of the right portion of the B string is needed to produce a LCS of length k with the last $(m-i+1)$ symbols of A . Not all $L_i(k)$ values are defined. $L_i^*(k)$ is analogous to $L_i(k)$.

Example. If $A = aadcbcad a$, ($m=9$) and $B = cabdacbc$ ($n=8$), $L_9(1) = 5$ because $A(9:9) = a$, and $B(5:8) = acbc$ have a LCS of length 1, and $A(9:9)$ and $B(i:8)$ do not have a LCS of length 1, for any $i > 5$. Similarly $L_7(2) = 4$, $L_1^*(1) = 2$, $L_3^*(2) = 4$.

The following three basic lemmas are similar to those in Nakatsu et al. [11]; their proofs are omitted.

Lemma 1. $\forall i \in [1, m]$, if $L_i(1), L_i(2), \dots, L_i(k)$ are defined, then $L_i(0) > L_i(1) > L_i(2) > \dots > L_i(k)$.

Lemma 2. $\forall i \in [1, m-1]$, $\forall j \in [1, m]$, if both $L_{i+1}(j)$, $L_i(j)$ are defined, then $L_{i+1}(j) \leq L_i(j)$.

Lemma 3. Let h be the largest number such that $A(i)=B(h)$ and $h < L_{i+1}(j-1)$. Then,

$$L_i(j) = \begin{cases} h & \text{if } L_{i+1}(j) \text{ is defined} \\ L_{i+1}(j) & \text{if } h \text{ does not exist} \\ \max(h, L_{i+1}(j)) & \text{if both are defined} \\ \text{undefined} & \text{otherwise.} \end{cases}$$

Analogous lemmas can be given for the $L_i^*(j)$ values but the following lemma implies them. $L_i(j)_{(A,B)}$ denotes that $L_i(j)$ values are calculated using the strings A and B . Throughout the rest of this paper, unless subscripted by a string pair, $L_i(j)$, $L_i^*(j)$ values are assumed to be calculated using the string pair (A, B) only.

Lemma 4. Assume that $L_i(j)$ values for the string pair $(\hat{A}, \hat{B})$ are known. The $L_i^*(j)$ values for the string pair (A, B) are given by

$$L_i^*(j)_{(A,B)} = n+1 - L_{m-i+1}(j)_{(\hat{A}, \hat{B})}$$

It should be emphasized again that $L_i^*(j)$ values are calculated using the strings A and B , while the $L_i(j)$ values are calculated using the strings $\hat{A}$ and $\hat{B}$.

Proof. Let $k = L_{m-i+1}(j)_{(\hat{A}, \hat{B})}$ be defined. This implies that $\hat{A}(m-i+1:m)$ and $\hat{B}(k:n)$ have a LCS of length j , and k is maximal. In other words, $A(1:i)$ and $B(1:n+1-k)$ have a LCS of length j , and k is maximal, or $n+1-k$ is minimal.

Hence, $L_i^*(j)_{(A,B)} = n+1 - L_{m-i+1}(j)_{(\hat{A}, \hat{B})}$.

The above lemma gives a method to compute $L_i^*(j)$ values, given a scheme to compute $L_i(j)$ values.

Definition 3. An ordered pair (u, v) is a valid cut, with respect to a LCS C , if there exist strings C_1, C_2 such that

$$\begin{aligned} C_1 &= \text{LCS}(A(1:u), B(1:v)), \\ C_2 &= \text{LCS}(A(u+1:m), B(v+1:n)) \end{aligned}$$

and

$$C = C_1 C_2.$$

For the example considered earlier, $(4, 6)$ is a valid cut with respect to the LCS “ $adcbc$ ” because there exist $C_1 = “adc”$, $C_2 = “bc”$ such that $C_1 = \text{LCS}(A(1:4), B(1:6))$, $C_2 = \text{LCS}(A(5:9), B(7:8))$ and $C = C_1 C_2$. On the other hand, $(5, 6)$ is not a valid cut.

Definition 4. Let C be an LCS of A and B . $W(A, B) = |A| - |C| = m - p$.

$W(A, B)$ is “waste” in A , that is, the number of symbols in A that are not in C .

Definition 5. An ordered pair (u, v) is a perfect cut if

- (i) (u, v) is a valid cut with respect to some LCS C of A and B .
- (ii) $W(A(1:u), B(1:v)) = [(m-p)/2]$.

Obviously, for a perfect cut, $W(A(u+1:m), B(v+1:n)) = [(m-p)/2]$. For the example considered previously, $(7, 8)$ is a perfect cut because it is a valid cut

and $W(A(1:7), B(1:8)) = 7 - 5 = 2 (= \lfloor (m-p)/2 \rfloor)$. There is no other perfect cut for the given strings in the example.

Theorem 1 below asserts the existence of a perfect cut for any two strings A and B ; Theorem 2 gives a characterisation of the perfect cut.

Theorem 1. *For any two strings A and B , a perfect cut exists.*

Proof. Let C be an arbitrary LCS of A and B . Let the symbols that were deleted from A to get C be

$$A(i'_1), A(i'_2), \dots, A(i'_{m-p}) \quad (i'_1 < i'_2 < \dots < i'_{m-p}).$$

The symbols of A that have not been deleted are

$$A(i_1), A(i_2), \dots, A(i_p) \quad (i_1 < i_2 < \dots < i_p).$$

The symbols of B that have not been deleted are

$$B(j_1), B(j_2), \dots, B(j_p) \quad (j_1 < j_2 < \dots < j_p).$$

Obviously $A(i_k) = B(j_k) = C(k)$ ($1 \leq k \leq p$).

$$u = i'_{\lfloor (m-p)/2 \rfloor},$$

and

$$v = j_r \quad \text{where } r \text{ is the largest } k \text{ such that } i_k < u.$$

Put $A_1 = A(1:u)$, $A_2 = A(u+1:m)$, $B_1 = B(1:v)$ and $B_2 = B(v+1:n)$.

Obviously, $C_1 = C(1)C(2)\dots C(r)$ is a common subsequence of A_1 and B_1 and $C_2 = C(r+1)C(r+2)\dots C(p)$ is a common subsequence of A_2 and B_2 . No string longer than r can be a common subsequence of A_1 and B_1 , because then that string catenated with C_2 would give a common subsequence of A and B longer than p . Therefore, $C_1 = \text{LCS}(A_1, B_1)$. Similarly, $C_2 = \text{LCS}(A_2, B_2)$. $C = C_1 C_2$. Hence, (u, v) is a valid cut.

$W(A_1, B_1) = \text{number of symbols that are deleted from } A_1, \text{ to get}$

$$C_1 = \lfloor (m-p)/2 \rfloor.$$

Hence, the perfect cut exists.

Let w and w' denote $\lfloor (m-p)/2 \rfloor$ and $\lfloor (m-p)/2 \rfloor$ respectively.

Theorem 2. *An ordered pair (u, v) is a perfect cut if and only if $L_{u+1}(m-u-w') > v \geq L_u^*(u-w)$.*

Proof. If the pair (u, v) is a perfect cut, then $|\text{LCS}(A(1:u), B(1:v))| = k$ for some k and $|\text{LCS}(A(u+1:m), B(v+1:n))| = p-k$. Also $W(A(1:u), B(1:v)) = w$. This implies that $L_u^*(k) \leq v$ and $L_{u+1}(p-k) \geq v+1$, or that $L_{u+1}(m-u-w') \geq v+1 > v \geq L_u^*(u-w)$.

Conversely, if $L_{u+1}(m-u-w') > v \geq L_u^*(u-w)$, then $A(1:u)$ and $B(1:L_u^*(u-w))$ have a LCS of length $u-w$, and $A(u+1:m)$ and $B(L_{u+1}(m-u-w'):n)$ have a LCS of length $m-u-w'$. Hence, $A(1:u)$ and $B(1:v)$ have a common subsequence of length $u-w$, and $A(u+1:m)$ and $B(v+1:n)$ have a common subse-

quence of length $m - u - w'$. Since $(u - w) + (m - u - w') = p$,

$$|\text{LCS}(A(1:u), B(1:v))| = u - w$$

and

$$|\text{LCS}(A(u+1:m), B(v+1:n))| = m - u - w'. \quad W(A(1:u), B(1:v)) = u - (u - w) = w.$$

Therefore, (u, v) is a perfect cut.

With these definitions and characterisation we are ready to present the algorithm.

3. Algorithm

Given A and B are strings of length m and n respectively, and p , the length of the LCS of A and B , algorithm LCS calculates the longest common subsequence of A and B . We assume that A and B are stored globally and that substrings of these strings can be passed as arguments by passing two numbers, namely the left and right end indices of the substring.

The divide and conquer technique is used in solving the problem. The algorithm finds a perfect cut for the string pair (A, B) , solves the subproblems of finding the LCS of A_1 and B_1 , and A_2 and B_2 , and combines the solutions.

LCS(A, B, m, n, p, C)

```

  if  $m - p < 2$ 
  1   then solve the base case
  2   else begin
  2.1   find a perfect cut  $(u, v)$ , for  $A$  and  $B$ ;
  2.2   LCS( $A(1:u), B(1:v), u, v, u - w, C_1$ );
        LCS( $A(u+1:m), B(v+1:n), m - u, n - v, m - u - w', C_2$ );
  2.3    $C = C_1 C_2$ 
  end
```

We now present the details of the following.

- (i) Step 1, or the base case.
- (ii) Step 2.1, or finding the perfect cut.
- (iii) Calculation of p , the length of the LCS.

It is convenient to define two procedures before we consider the solutions for the above. From Theorem 2, we can see that we need all the $L_i(j)$ values such that $i + j = w + 1$. Procedure calmid calculates these required values. It repeatedly calls procedure fillone, which calculates all $L_i(j)$ values such that $i + j = s + 1$ given all the $L_i(j)$ values such that $i + j = s + 2$. Lemma 3 is used by procedure fillone to calculate the $L_i(j)$ values such that $i + j = s + 1$ given all the $L_i(j)$ values such that $i + j = s + 2$. Also, by taking a different view of procedure fillone, as we shall see in the proof of correctness of length, we can use it to calculate the length of the LCS. Procedure fillone also turns out to be useful in solving the base case.

Procedure Fillone

We use two arrays $R1(0:n)$ and $R2(0:n)$ to store the given and the calculated values respectively. Assume $m \geq s \geq \text{length of the LCS of } A \text{ and } B$. Given $L_{s+1}(1), L_s(2), L_{s+1}(3), \dots$, in the array elements $R1(1), R1(2), R1(3), \dots$, the procedure calculates $L_{s+1}(0) (=n+1), L_s(1), L_{s-1}(2), \dots$ and returns them in the array elements $R2(0), R2(1), R2(2), \dots$. That is, given all defined $L_{s+2-j}(j)$ values ($j > 1$), in $R1(j)$, all defined $L_{s+1-j}(j)$ values ($j > 0$) are calculated and returned in $R2(j)$. Variable r is used to indicate the largest k such that $L_{s+2-k}(k)$ is defined. At the end of the computation, r is updated to indicate the largest k such that $L_{s+1-k}(k)$ is defined.

```

fillone( $A, b, m, n, R1, R2, r, s$ )
begin
1.  $j := 1; i := s;$ 
    $over := false; R2(0) := n + 1;$ 
2. while  $i > 0$  and not  $over$  do
   begin
2.1 if  $j > r$  then  $lower\ B := 0$  else  $lower\ B := R1(j);$ 
      $pos\ B := R2(j - 1) - 1;$ 
2.2 while  $pos\ B > lower\ B$  and  $A(i) \neq B(pos\ B)$  do
      $pos\ B := pos\ B - 1;$ 
2.3  $temp := \max(pos\ B, lower\ B);$ 
     if  $temp = 0$  then  $over := true$ 
     else begin
        $R2(j) := temp;$ 
        $i := i - 1; j := j + 1$ 
     end
   end;
3.  $r = j - 1$ 
end

```

Proof of Correctness of Fillone

This procedure uses Lemma 3 to calculate $L_i(j)$ values in the steps 2.2 and 2.3. The scan for $A(i)$ (in the string B) starts from $L_{i+1}(j-1)$ and should end before it crosses $L_{i+1}(j)$, lower B is used to ensure that the scan does not cross over $L_{i+1}(j)$. If $L_{i+1}(j)$ is undefined, lower B is set to 0 and it ensures that the scan stops when the left end of the string is reached. Boolean variable “over” is used to terminate the loop in step 2 when no more of the required $L_i(j)$ values can be calculated.

$L_{s+1}(0)$ is calculated, in i and j set to s and 1 respectively (in step 1). In each iteration of the loop in step 2, $L_i(j)$ is calculated (in steps 2.2 and 2.3) and i decremented and j incremented, so that in the following iterations, only the required $L_i(j)$ values are calculated. When the loop in step 2 terminates, j will be one more than the largest k such that $L_{s+1-k}(k)$ is defined, because $L_i(j)$ is undefined. Hence step 3 is also correct.

Time-Space Analysis of Fillone

In step 2.2, all the symbols of string B are compared sequentially from the last symbol to the first symbol. Hence step 2.2 is executed at most n times. It is obvious that other steps are not executed more than n times. Hence, time complexity is $O(n)$. The procedure uses only linear arrays $R1$, $R2$, so the space complexity is $O(n + m)$.

Procedure Calmid

This procedure calculates $L_{m-x+1-j}(j)$ values and store them in $LL(j)$ ($j > 0$). r is set to the largest j such that $L_{m-x+1-j}(j)$ is defined.

```

calmid( $A, B, m, n, x, LL, r$ )
begin
1.  $r := 0$ ;
2. for  $s := m$  downto  $m - x$  do
   begin
      $fillone(A, B, m, n, R1, R2, r, s)$ ;
      $R1(j) := R2(j)$  [ $j := 0 \dots r$ ]
   end;
3.  $LL(j) := R1(j)$  [ $j := 0 \dots r$ ]
end

```

Proof of Correctness of Calmid. At the end of the i^{th} iteration, $R1(j)$ contains $L_{m-i+2-j}(j)$ ($0 \leq j \leq r$). In the next iteration, the call to $fillone$ will store $L_{m-i+1-j}(j)$ in $R2(j)$. Subsequent copying of $R2(j)$ into $R1(j)$ ensures that at the end of the $(i+1)^{\text{th}}$ iteration, $R1(j)$ contains $L_{m-(i+1)-j+2}(j)$ ($0 \leq j \leq r$) (Note that r is updated in the call to $fillone$.)

Hence, at the end of the $(x+1)^{\text{th}}$ iteration, $R1(j)$ contains $L_{m-x-j+1}(j)$, which is copied into $LL(j)$ ($0 \leq j \leq r$).

Time-Space Analysis of Calmid

The loop body is executed exactly $(x+1)$ times. In each iteration of the loop, a call to $fillone$ and copying of $R2(j)$ into $R1(j)$ is done, which take $O(n)$ and $O(r)$ ($< O(n)$) time respectively. Hence, the time complexity is $O(n(x+1))$. The procedure uses only linear arrays $R1$, $R2$, LL , so the space complexity is $O(n + m)$.

Now, we will describe the three incomplete portions of the algorithm.

(i) **Step 1, or the base case:**

```

1.  $calmid(A, B, m, n, m - p, LL, r)$ ;
    $i := 1$ ;

```

```

2. while  $i \leq p$  and  $A(i) = B(LL(p-i+1))$  do
  begin
     $C(i) := A(i)$ ;
     $i := i + 1$ 
  end;
3.  $i := i + 1$ ;
4. while  $i \leq m$  do
  begin
     $C(i-1) := A(i)$ ;
     $i := i + 1$ 
  end

```

When $p = m$, the statements in step 4 are not executed at all and A is copied into C in step 2. When $p = m - 1$, the step 3 can be seen as the statement that drops out one character from A to get C . It is easy to verify that the wrong character is not dropped.

(ii) **Step 2.1, or finding the perfect cut:**

```

2.1.1  $\text{calmid}(\hat{A}, \hat{B}, m, n, w, LL1, r_1)$ ;
       $LL1(j) := n + 1 - LL1(j) \ [j := 0 \dots r_1]$ ;
2.1.2  $\text{calmid}(A, B, m, n, w', LL2, r_2)$ ;
2.1.3  $u := k + w$ ;  $k \leq r_1$  and  $p - k \leq r_2$  and  $LL1(k) < LL2(p - k)$ ;
       $v := LL1(k)$ 

```

Proof of Correctness (That The Above Steps Calculate a Perfect Cut)

At the end of step 2.1.1, $n + 1 - L_{m-w+1-k}(k)_{(\hat{A}, \hat{B})}$ values are available in $LL1(k)$ ($0 \leq k \leq r_1$). By lemma 4, $L_{w+k}^*(k)$ values are available in $LL1(k)$ ($0 \leq k \leq r_1$). In step 2.1.2, $L_{w+k+1}(m - (w + k) - w')$ ($= L_{m-w'+1-(p-k)}(p - k)$) values are stored in $LL2(p - k)$, ($0 \leq p - k \leq r_2$). Putting $u = k + w$, we see that the choice of u and v in step 2.1.3 satisfies the conditions imposed in Theorem 2. Hence, (u, v) calculated above is a perfect cut.

(iii) Calculation of p :

```

   $\text{length}(A, B, m, n, p)$ ;
  begin
1.    $r := 0$ ;  $s := m + 1$ ;
2.   while  $s > r$  do
    begin
       $s := s - 1$ ;
       $\text{fillone}(A, B, m, n, R1, R2, r, s)$ ;
       $R1(j) := R2(j) \ [j := 0 \dots r]$ 
    end;
3.    $p := s$ 
  end

```


Proof of Correctness of Length

The values stored by *fillone* in *R2* are not used directly. Each call to *fillone* can be seen as an attempt to calculate $L_s(1)$. If $L_s(1)$ is defined, then the loop terminates immediately afterwards because then $r=s$. The order of calls to *fillone* (with $s=m, m-1, \dots$) ensures that the largest s such that $L_s(1)$ is defined is found, and this is the length of the LCS.

Time-Space Analysis of Length

If the length of the LCS is p , the statements in step 2 are executed exactly $(m-p+1)$ times for $s=m+1, \dots, p+1$. The statements in step 2 take $O(n)$ time. Hence, the time required to calculate p is $O(n(m-p))$. Again, only linear arrays *R1*, *R2* are used, so the space complexity is $O(n+m)$.

Now, we will give the proof of correctness and the time-space complexity of LCS.

Proof of Correctness of LCS

If the problem is trivial, step 1 correctly calculates the LCS. If the problem is not trivial, step 2.1 correctly finds a perfect cut (proof of correctness for step 2.1). So there must exist solutions C_1 and C_2 to the subproblems $(A(1:u), B(1:v))$ and $(A(u+1:m), B(v+1:n))$ respectively such that $\text{LCS}(A, B) = C_1 C_2$.

Time-Space Analysis of LCS

We represent the time complexity of the problem of computation of the LCS of the strings A and B of length m and n , respectively, by $T(m, n, p)$, where p is the length of the LCS. Obviously $T(m, n, p)$, when $m-p < 2$, is $O(n+m)$ because a call to *calmid* with $x=m-p$ is made and additional operations on the vector A are done.

When $m-p \geq 2$, $T(m, n, p)$ equals the cost of the two calls to *calmid*, the cost of operations on linear vectors, and the cost of solving the subproblems. The cost of the two calls to *calmid* and the operations on the linear vectors will be

$$\begin{aligned} &O(n(w+1)) + O(n(w'+1)) + O(n+m) \\ &= O(n(m-p+2)) + O(n+m) \\ &= O(n(m-p)). \end{aligned}$$

Therefore,

$$T(m, n, p) = O(n(m-p)) + T(u, v, u-w) + T(m-u, n-v, m-u-w'),$$

where (u, v) is a perfect cut.

Since $u - (u - w) = w = \lceil (m - p)/2 \rceil$, and $m - u - (m - u - w') = w' = \lceil (m - p)/2 \rceil$, $T(m, n, p)$ can be seen to be $O(n(m - p))$.

The time required to calculate p is also $O(n(m - p))$, (Time-space analysis of length), so the overall time complexity for the calculation of the LCS is $T(m, n, p) + O(n(m - p)) = O(n(m - p))$.

Linear storage is required to store the vectors C_1 , C_2 , C , $LL1$, and $LL2$. Recall that for every call we do not pass the entire substring of A or B but only the left and right end indices. Since the maximum depth of recursion at any time is $O(\log(m - p))$, the space complexity of algorithm LCS remains linear.

From the space analysis of all the procedures, we infer that they require only $O(n + m)$ storage. In addition, common storage required to store the strings A and B is also linear. Hence, the solution presented in this paper has linear space complexity.

Acknowledgements. The authors would like to thank the referees for their helpful suggestions.

References

1. Aho, A.V., Hirschberg, D.S., Ullmann, J.D.: Bounds on the complexity of the longest common subsequence problem. *JACM* **23**, 1–12 (1976)
2. Dayhoff, M.O.: Computer analysis of protein evolution. *Scientific American* **221**, 86–95 (1969)
3. Hirschberg, D.S.: A linear space algorithm for computing maximal common subsequences. *Commun. ACM* **18**, 341–343 (1975)
4. Hirschberg, D.S.: Algorithms for the longest common subsequence problem. *JACM* **24**, 664–675 (1977)
5. Hsu, W.J., Du, M.W.: New algorithms for the LCS problem. *J. Comput. Syst. Sci.* **29**, 133–152 (1984)
6. Hunt, J.W., Szymanski, T.G.: A fast algorithm for computing longest common subsequences. *Commun. ACM* **20**, 350–353 (1977)
7. Lu, S.Y., Fu, K.S.: A sentence-to-sentence clustering procedure for pattern analysis. *IEEE Trans. Syst. Man. Cybernet.* **SMC-8**, 381–389 (1978)
8. Maier, D.: The complexity of some problems on subsequences and supersequences. *JACM* **25**, 322–336 (1978)
9. Masek, W.J., Paterson, M.S.: A faster algorithm for computing string edit distances. *J. Comput. Syst. Sci.* **20**, 18–31 (1980)
10. Mukhopadhyay, A.: A fast algorithm for the longest-common-subsequence problem. *Inf. Sci.* **20**, 69–82 (1980)
11. Nakatsu, N., Kambayashi, Y., Yajima, S.: A longest common subsequence algorithm suitable for similar text strings. *Acta Inf.* **18**, 171–179 (1982)
12. Wagner, R.A., Fischer, M.J.: The string-to-string correction problem. *JACM* **21**, 168–173 (1974)

Received March 24, 1986/February 17, 1987